



**TẬP ĐOÀN CÔNG NGHIỆP – VIỄN THÔNG QUÂN ĐỘI**

## **BÁO CÁO MINI-PROJECT**

***ENGINE THỰC THI AI AGENT CHO NỀN TẢNG  
VẬN HÀNH MẠNG***

*Vũ Cao Phong*

*Phvu1403@gmail.com*

**Chương trình Viettel Digital Talent 2026**

**Lĩnh vực: Software Engineer**

**Mentor:** Vũ Hữu Nam Anh

**Đơn vị:** VTNet

# LỜI MỞ ĐẦU

Trí tuệ nhân tạo ngày càng được ứng dụng nhiều trong vận hành hạ tầng viễn thông. Đáng chú ý là các mô hình ngôn ngữ lớn gần đây đã có khả năng *gọi công cụ*: thay vì chỉ sinh văn bản, mô hình có thể tự truy xuất dữ liệu và thao tác trên các hệ thống bên ngoài. Khả năng này cho phép kỹ sư vận hành làm việc với hạ tầng bằng ngôn ngữ tự nhiên, thay cho việc thao tác trực tiếp trên nhiều hệ thống kỹ thuật rời rạc. Tại một nhà mạng quy mô lớn, nơi sự cố cần được xử lý nhanh và đòi hỏi độ tin cậy cao, một trợ lý AI cho Trung tâm Điều hành Mạng (NOC) có thể giúp rút ngắn thời gian xử lý sự cố và giảm phụ thuộc vào kinh nghiệm cá nhân.

Báo cáo này được thực hiện trong khuôn khổ chương trình Viettel Digital Talent 2026, trình bày quá trình nghiên cứu, thiết kế và triển khai đề tài **Engine thực thi AI Agent cho nền tảng vận hành mạng**, chịu trách nhiệm chạy tác tử và gọi công cụ trong một nền tảng phục vụ vận hành mạng viễn thông. Mục đích của báo cáo là ghi lại các quyết định kiến trúc của hệ thống, cách hiện thực hoá từng thành phần và đánh giá mức độ hoàn thành so với thiết kế. Báo cáo vừa là tài liệu kỹ thuật phục vụ phát triển tiếp, vừa ghi lại kết quả học tập của em qua dự án. Em xin trân trọng cảm ơn Mentor và chương trình Viettel Digital Talent 2026 đã định hướng và đồng hành trong suốt quá trình thực hiện.

# TÓM TẮT

**Tóm tắt:** Báo cáo trình bày quá trình nghiên cứu, thiết kế và triển khai đề tài **Engine thực thi AI Agent cho nền tảng vận hành mạng**, phần lõi của một nền tảng AI Agent phục vụ vận hành mạng viễn thông trong khuôn khổ chương trình Viettel Digital Talent 2026. Engine nhận câu hỏi của kỹ sư vận hành, chạy một vòng lặp gọi công cụ trên nền LangGraph gồm hai nút think/act cùng một nút chặn vòng lặp, gọi các *skill* viễn thông để truy vấn dữ liệu thật (cơ sở dữ liệu thiết bị, cảnh báo và lệnh chẩn đoán qua SSH), rồi trả kết quả về giao diện theo thời gian thực bằng giao thức SSE với một bộ sự kiện thống nhất gồm mười hai loại. Toàn bộ phiên hội thoại, lời gọi công cụ, đối số và kết quả được lưu trong PostgreSQL dưới dạng JSONB để phục vụ cho việc xem lại, đồng thời được truy vết quan sát qua Langfuse. Hệ thống tách bạch năm thành phần (vòng lặp agent, trườ tượng nhà cung cấp LLM, mô hình skill, tầng streaming và tầng lưu trữ). Kết quả bước đầu cho thấy luồng xử lý đầu cuối (từ câu hỏi tới streaming câu trả lời kèm lời gọi công cụ trên dữ liệu thật) đã hoạt động, cho thấy thiết kế là khả thi. Các hạng mục chưa hoàn thiện được nêu trung thực cùng hướng phát triển tiếp theo.

**Từ khóa:** *AI Agent, tool-calling, LangGraph, streaming SSE, vận hành mạng viễn thông.*

# MỤC LỤC

|                                                       |            |
|-------------------------------------------------------|------------|
| <b>Lời mở đầu</b>                                     | <b>i</b>   |
| <b>Tóm tắt</b>                                        | <b>ii</b>  |
| <b>DANH MỤC HÌNH ẢNH</b>                              | <b>v</b>   |
| <b>DANH MỤC BẢNG BIỂU</b>                             | <b>vi</b>  |
| <b>DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT</b>              | <b>vii</b> |
| <b>1 Giới thiệu</b>                                   | <b>1</b>   |
| 1.1 Đặt vấn đề . . . . .                              | 1          |
| 1.2 Bối cảnh dự án . . . . .                          | 2          |
| 1.3 Thách thức . . . . .                              | 2          |
| 1.4 Mục tiêu và phạm vi . . . . .                     | 3          |
| 1.5 Yêu cầu hệ thống . . . . .                        | 3          |
| 1.6 Đóng góp chính . . . . .                          | 5          |
| 1.7 Bố cục báo cáo . . . . .                          | 5          |
| <b>2 Nội dung và phương pháp</b>                      | <b>6</b>   |
| 2.1 Nền tảng lý thuyết . . . . .                      | 6          |
| 2.1.1 Mô hình ngôn ngữ lớn và tool-calling . . . . .  | 6          |
| 2.1.2 Tác tử và vòng lặp suy nghĩ–hành động . . . . . | 6          |
| 2.1.3 LangGraph . . . . .                             | 7          |
| 2.1.4 Truyền sự kiện thời gian thực với SSE . . . . . | 7          |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 2.1.5    | Quan sát hệ thống LLM . . . . .                               | 7         |
| 2.2      | Các hướng tiếp cận hiện tại . . . . .                         | 8         |
| 2.3      | Khảo sát và lựa chọn công nghệ . . . . .                      | 9         |
| 2.4      | Kiến trúc tổng thể . . . . .                                  | 12        |
| 2.5      | Đặc tả giao diện lập trình (API) . . . . .                    | 13        |
| 2.6      | Vòng lặp thực thi agent . . . . .                             | 14        |
| 2.7      | Trình tượng nhà cung cấp LLM . . . . .                        | 16        |
| 2.8      | Mô hình skill . . . . .                                       | 17        |
| 2.9      | Giao thức streaming: envelope sự kiện và Translator . . . . . | 18        |
| 2.10     | Lưu trữ và kiểm toán . . . . .                                | 21        |
| 2.11     | Quan sát và truy vết với Langfuse . . . . .                   | 21        |
| <b>3</b> | <b>Kết quả thực hiện và đánh giá</b>                          | <b>23</b> |
| 3.1      | Mục tiêu thực nghiệm . . . . .                                | 23        |
| 3.2      | Môi trường và cách triển khai . . . . .                       | 23        |
| 3.3      | Kết quả luồng đầu cuối . . . . .                              | 24        |
| 3.4      | Đánh giá . . . . .                                            | 24        |
| <b>4</b> | <b>Kết luận</b>                                               | <b>25</b> |
|          | <b>Tài liệu tham khảo</b>                                     | <b>27</b> |

# DANH MỤC HÌNH ẢNH

|     |                                                                                                                                                       |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1 | Kiến trúc tổng thể . . . . .                                                                                                                          | 12 |
| 2.2 | Đồ thị LangGraph của vòng lặp agent . . . . .                                                                                                         | 14 |
| 2.3 | Máy trạng thái cấp lượt (turn-level FSM). . . . .                                                                                                     | 19 |
| 2.4 | Lược đồ dữ liệu chính phục vụ xem lại hội thoại. . . . .                                                                                              | 21 |
| 2.5 | Phân cấp truy vết trên Langfuse: trace $\rightarrow$ span $\rightarrow$ sub-span,<br>song song với lược đồ phiên/lượt/lời gọi công cụ ở Hình 2.4. . . | 22 |

# DANH MỤC BẢNG BIỂU

|     |                                                      |    |
|-----|------------------------------------------------------|----|
| 1.1 | Yêu cầu chức năng (Functional Requirements).         | 4  |
| 1.2 | Yêu cầu phi chức năng (Non-Functional Requirements). | 4  |
| 2.1 | Lựa chọn công nghệ.                                  | 9  |
| 2.2 | Các endpoint HTTP của engine.                        | 13 |
| 2.3 | Tham số của /api/chat và /api/chat/stream.           | 14 |
| 2.4 | Bảng chân trị của hàm định tuyến should_continue.    | 15 |
| 2.5 | Hai loại skill trong mô hình.                        | 17 |
| 2.6 | Mười hai loại sự kiện trong envelope SSE.            | 18 |

# DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

| Viết tắt / Thuật ngữ | Tiếng Anh đầy đủ                | Ý nghĩa                                                                 |
|----------------------|---------------------------------|-------------------------------------------------------------------------|
| Agent                | Agent                           | Tác tử: chương trình dùng LLM để lập suy nghĩ–hành động tới khi trả lời |
| Tool-calling         | Tool-calling / Function calling | LLM trả về lời gọi hàm có cấu trúc (tên + đối số) thay vì văn bản tự do |
| NOC                  | Network Operations Center       | Trung tâm Điều hành Mạng                                                |
| Skill                | Skill                           | Công cụ agent có thể gọi (truy vấn CSDL, chạy lệnh SSH, gọi URL...)     |
| LLM                  | Large Language Model            | Mô hình ngôn ngữ lớn                                                    |
| SSE                  | Server-Sent Events              | Giao thức truyền sự kiện một chiều server → client trên nền HTTP        |
| FSM                  | Finite State Machine            | Máy trạng thái hữu hạn                                                  |
| Envelope             | Event envelope                  | Khung JSON thống nhất bọc mỗi sự kiện stream                            |
| Provider             | LLM Provider                    | Nhà cung cấp LLM (Anthropic, Groq...) kế thừa lớp trừu tượng chung      |
| JSONB                | JSON Binary                     | Kiểu JSON nhị phân của PostgreSQL, truy vấn/đánh chỉ mục được           |
| Replay / Audit       | Replay / Audit                  | Tái dựng và kiểm toán lại một phiên hội thoại từ dữ liệu đã lưu         |
| Observability        | Observability                   | Khả năng quan sát, truy vết hệ thống                                    |



# Chương 1

## Giới thiệu

### 1.1. Đặt vấn đề

Vận hành một mạng viễn thông quy mô lớn đòi hỏi kỹ sư tại Trung tâm Điều hành Mạng (NOC) phải làm việc đồng thời trên nhiều hệ thống rời rạc: kho thiết bị, kho cảnh báo, hệ đo lường chỉ số chất lượng, và truy cập trực tiếp vào từng node qua SSH để chẩn đoán. Mỗi hệ thống có giao diện và ngôn ngữ truy vấn riêng và khi có sự cố xảy ra, áp lực thời gian khiến việc ghép nối thủ công các nguồn dữ liệu này trở thành điểm nghẽn, phụ thuộc nhiều vào kinh nghiệm cá nhân và dễ sai sót.

Các mô hình ngôn ngữ lớn hiện đại đã có khả năng *gọi công cụ*: thay vì chỉ sinh văn bản, mô hình có thể trả về một lời gọi hàm có cấu trúc để hệ thống thực thi, nhận lại kết quả rồi tiếp tục suy luận [4, 5]. Cơ chế này mở ra khả năng xây dựng một *trợ lý vận hành* hiểu câu hỏi tiếng Việt, tự quyết định gọi công cụ nào và theo trình tự nào để truy xuất dữ liệu thật, sau đó tổng hợp thành câu trả lời ngắn gọn cho kỹ sư.

Tuy nhiên, bản thân LLM chỉ là một thành phần. Để một trợ lý như vậy dùng được trong môi trường vận hành sẽ cần một **engine thực thi** đảm nhận: điều phối vòng lặp suy nghĩ–hành động và chặn vòng lặp vô hạn; gọi công cụ một cách an toàn, có giới hạn thời gian và xử lý lỗi; phát triển trình theo thời gian thực để người dùng theo dõi được agent đang suy luận và gọi skill nào; và lưu vết đầy đủ mọi bước để có thể tái dựng và kiểm toán về sau. Đây chính là phạm vi của đề tài **Engine thực thi AI Agent cho nền tảng vận hành mạng** mà báo cáo này trình bày.

## 1.2. Bối cảnh dự án

Đề tài Engine thực thi AI Agent cho nền tảng vận hành mạng được thực hiện trong khuôn khổ chương trình Viettel Digital Talent 2026, hướng tới một nền tảng AI Agent phục vụ vận hành viễn thông gồm nhiều module. Trong đó, **Module 1** là engine thực thi — phần lõi chịu trách nhiệm chạy agent và gọi công cụ; **Module 2** quản lý cấu hình agent và **Module 3** là kho định nghĩa skill. Theo định hướng đã thống nhất với Mentor, Module 2 và Module 3 đóng vai trò *đầu vào* cho Module 1 dưới dạng file cấu hình hoặc dữ liệu, chưa yêu cầu tích hợp API giữa các module trong giai đoạn này. (xem Mục 2.4).

## 1.3. Thách thức

Việc xây dựng một engine thực thi agent dùng được trong vận hành đặt ra nhiều thách thức kỹ thuật:

- **Phản hồi thời gian thực.** Người vận hành cần thấy tiến trình ngay khi agent suy luận và gọi công cụ, thay vì chờ tới khi có câu trả lời cuối. Điều này đòi hỏi một cơ chế truyền streaming phân biệt được nhiều loại tín hiệu như suy luận, câu trả lời, lời gọi công cụ trong cùng một kết nối.
- **Hỗ trợ nhiều mô hình LLM.** Mỗi nhà cung cấp (Anthropic, OpenAI, Groq...) có định dạng tool-calling và streaming khác nhau. Engine cần một lớp trừu tượng để thay đổi nhà cung cấp mà không sửa vòng lặp lỗi.
- **Gọi công cụ thật một cách an toàn.** Công cụ không phải là hàm thuần mà là truy vấn cơ sở dữ liệu, lệnh SSH trên node sống... Engine phải kiểm tra đối số, đặt giới hạn thời gian, xử lý lỗi và đặc biệt cẩn trọng với các thao tác có khả năng gây tác dụng phụ.
- **Chặn vòng lặp.** Một agent có thể rơi vào vòng lặp gọi công cụ vô hạn nên cần một cơ chế chặn số vòng rõ ràng.
- **Xem lại hội thoại** Trong môi trường vận hành, mọi quyết định của agent và mọi lời gọi công cụ phải lưu lại được để phục vụ cho việc xem lại các

hành động trong quá khứ.

- **Cô lập với module chưa sẵn sàng.** Module 2/3 chưa hoàn thiện; engine phải hoạt động độc lập nhưng vẫn sẵn sàng tích hợp về sau mà không cần thiết kế lại.

## 1.4. Mục tiêu và phạm vi

Mục tiêu của dự án là **thiết kế và hiện thực hoá một engine thực thi agent cho bài toán vận hành mạng viễn thông**, cụ thể:

1. Xây dựng vòng lặp tool-calling chạy được với công cụ thật, có chặn vòng lặp và xử lý lỗi.
2. Trừu tượng hoá nhà cung cấp LLM để hỗ trợ nhiều mô hình khác nhau.
3. Thiết kế giao thức streaming SSE với bộ sự kiện thống nhất để truyền suy luận, câu trả lời và lời gọi công cụ theo thời gian thực.
4. Thiết kế mô hình skill cho phép định nghĩa và thực thi công cụ viễn thông.
5. Lưu trữ phiên hội thoại phục vụ xem lại và truy vết quan sát qua Langfuse.

**Phạm vi.** Báo cáo tập trung vào engine thực thi (vòng lặp, dispatcher skill, error/timeout), lớp trừu tượng đa provider, streaming SSE, lưu trữ phiên và quan sát, cùng một giao diện chat tối thiểu để minh hoạ. Việc tích hợp API với Module 2/3 nằm *ngoài phạm vi* do hai module này hiện đang được lưu dưới dạng các file cấu hình

## 1.5. Yêu cầu hệ thống

Từ mục tiêu và phạm vi trên, yêu cầu của engine được cụ thể hoá thành yêu cầu chức năng (Bảng 1.1) và phi chức năng (Bảng 1.2).

Bảng 1.1: Yêu cầu chức năng (Functional Requirements).

| Mã    | Mô tả                                                                                                               |
|-------|---------------------------------------------------------------------------------------------------------------------|
| FR-01 | Tiếp nhận câu hỏi ngôn ngữ tự nhiên và chạy vòng lặp tool-calling tới câu trả lời cuối.                             |
| FR-02 | Tự chọn và gọi skill phù hợp (truy vấn kho thiết bị, kiểm tra service qua SSH, skill phức hợp) để lấy dữ liệu thật. |
| FR-03 | Chặn vòng lặp ở ngưỡng MAX_ITER để tránh treo và bảo vệ chi phí gọi LLM.                                            |
| FR-04 | Phát tiến trình theo thời gian thực (đổi trạng thái, câu trả lời, vòng đời lời gọi công cụ) qua SSE.                |
| FR-05 | Cho phép chọn nhà cung cấp và mô hình LLM theo từng yêu cầu.                                                        |
| FR-06 | Lưu phiên, tin nhắn và lời gọi công cụ (bao gồm cả đối số và kết quả) phục vụ xem lại hội thoại trong quá khứ.      |
| FR-07 | Liệt kê phiên gần đây và tái dựng lịch sử một phiên đã lưu để hội thoại nhiều lượt.                                 |
| FR-08 | Báo lỗi hoặc quá thời gian của công cụ về giao diện thay vì làm hỏng cả phiên.                                      |

Bảng 1.2: Yêu cầu phi chức năng (Non-Functional Requirements).

| Mã     | Mô tả                                                                                                              |
|--------|--------------------------------------------------------------------------------------------------------------------|
| NFR-01 | Phản hồi thời gian thực: câu trả lời cuối không bị loại bỏ kể cả khi hàng đợi nghẽn.                               |
| NFR-02 | Bất đồng bộ: chạy async để một tiến trình phục vụ nhiều phiên streaming đồng thời.                                 |
| NFR-03 | Độc lập nhà cung cấp: đổi provider LLM không phải sửa vòng lặp lỗi.                                                |
| NFR-04 | Khả năng kiểm toán: đối số và kết quả công cụ lưu nguyên vẹn dạng JSONB.                                           |
| NFR-05 | Quan sát được: mỗi phiên/lượt/lời gọi công cụ gắn vết qua Langfuse; thiếu Langfuse thì suy giảm chứ không gây lỗi. |
| NFR-06 | An toàn cấu hình: khoá API và thông tin kết nối nạp từ biến môi trường, không nhúng trong mã nguồn.                |
| NFR-07 | Tái lập môi trường: khoá phiên bản phụ thuộc bằng (uv) và dựng hạ tầng bằng Docker Compose.                        |

## 1.6. Đóng góp chính

Những đóng góp chính của dự án gồm:

- Một kiến trúc engine tách bạch rõ ràng giữa vòng lặp agent, lớp trừu tượng nhà cung cấp LLM, mô hình skill, tầng streaming và tầng lưu trữ.
- Một giao thức sự kiện (envelope SSE gồm 12 loại) cùng một bộ *Translator* dịch sự kiện nội bộ của LangGraph sang envelope.
- Một mô hình skill lai: skill atomic được định nghĩa bằng YAML và thực thi trực tiếp qua các executor, cùng skill composite được định nghĩa bằng tệp SKILL.md và chạy qua orchestrator script.
- Một lược đồ dữ liệu phục vụ tái dựng, phân tầng độ bền cho dữ liệu giao dịch và dữ liệu best-effort.

## 1.7. Bố cục báo cáo

Phần còn lại của báo cáo được tổ chức như sau. **Chương 2** trình bày nền tảng lý thuyết, khảo sát lựa chọn công nghệ và toàn bộ kiến trúc cùng phương pháp hiện thực hoá engine. **Chương 3** mô tả môi trường, kết quả luồng đầu cuối và đánh giá so với mục tiêu. **Chương 4** tổng hợp hiện trạng hiện thực hoá so với thiết kế, nêu kết luận và hướng phát triển.

# Chương 2

## Nội dung và phương pháp

Chương này lần lượt trình bày: nền tảng lý thuyết liên quan (Mục 2.1); các hướng tiếp cận hiện tại (Mục 2.2); khảo sát và lựa chọn công nghệ (Mục 2.3); kiến trúc tổng thể của engine (Mục 2.4); đặc tả giao diện lập trình (Mục 2.5); và chi tiết phương pháp hiện thực hoá từng thành phần: vòng lặp thực thi (Mục 2.6), trù tượng nhà cung cấp LLM (Mục 2.7), mô hình skill lai (Mục 2.8), giao thức streaming (Mục 2.9), lưu trữ và kiểm toán (Mục 2.10) và quan sát hệ thống (Mục 2.11).

### 2.1. Nền tảng lý thuyết

#### 2.1.1. Mô hình ngôn ngữ lớn và tool-calling

Tool-calling là cơ chế cho phép LLM trả về một lời gọi hàm có cấu trúc gồm tên công cụ và đối số tuân theo một lược đồ JSON Schema được khai báo trước. Hệ thống thực thi công cụ đó, đưa kết quả trở lại cho LLM tiếp tục suy luận. Cả Anthropic và OpenAI đều cung cấp endpoint tool-calling chuẩn của nhà cung cấp [4, 5]; việc dùng cơ chế native ổn định hơn nhiều so với tự phân tích JSON từ văn bản.

#### 2.1.2. Tác tử và vòng lặp suy nghĩ–hành động

Một agent tool-calling vận hành theo vòng lặp: LLM *suy nghĩ* và có thể gọi công cụ; hệ thống *hành động* bằng cách thực thi công cụ; kết quả được đưa lại cho LLM ở vòng kế tiếp; quá trình lặp cho tới khi LLM đưa ra câu trả lời cuối không kèm lời gọi nào. Mô hình tư duy này kế thừa tinh thần của ReAct [1] —

đan xen suy luận và hành động. Điểm mấu chốt khi đưa vào môi trường thật là phải giới hạn số vòng lặp tối đa để tránh treo và kiểm soát chi phí gọi LLM.

### 2.1.3. LangGraph

LangGraph [2] mô hình hoá agent thành một đồ thị có hướng gồm các *nút* thao tác trên một *trạng thái* dùng chung, nối với nhau bằng các *cạnh*, trong đó cạnh điều kiện quyết định bước kế tiếp. Quan trọng hơn, LangGraph cung cấp API `astream_events()` phát ra các sự kiện (bắt đầu/kết thúc gọi mô hình, từng token được stream, bắt đầu/kết thúc công cụ) — đây là nền tảng để dựng tầng streaming thời gian thực.

### 2.1.4. Truyền sự kiện thời gian thực với SSE

Server-Sent Events (SSE) [6] là giao thức truyền sự kiện một chiều từ server tới client trên nền HTTP. Với bài toán này, luồng dữ liệu vốn một chiều (người dùng không gửi thêm dữ liệu trong khi agent đang xử lý một lượt), nên SSE là lựa chọn đủ dùng và nhẹ hơn WebSocket. SSE giúp giữ nguyên ngữ nghĩa HTTP, đi qua proxy dễ dàng và hiện thực đơn giản. Mỗi sự kiện SSE mang một khung JSON ở trường `'data:'` và lưu tên sự kiện ở trường `'event:'` để phía client có thể lắng nghe theo từng loại.

### 2.1.5. Quan sát hệ thống LLM

Ứng dụng LLM khó gỡ lỗi vì hành vi phụ thuộc vào dữ liệu vào/ra của mô hình qua nhiều vòng. Langfuse [8] cho phép gắn mỗi phiên thành một *trace*, mỗi lượt là một *span*, và mỗi lời gọi công cụ là một sub-span — phục vụ phân tích chi phí, độ trễ và truy vết lỗi của agent.

## 2.2. Các hướng tiếp cận hiện tại

Có nhiều cách hiện thực một tác tử tool-calling; lựa chọn ảnh hưởng trực tiếp tới ba yêu cầu cốt lõi của bài toán vận hành đó là: kiểm soát vòng lặp, streaming thời gian thực và khả năng dừng lại hội thoại trong quá khứ.

**Tự gọi SDK của nhà cung cấp.** Cách trực tiếp nhất là tự viết vòng lặp bằng SDK riêng của Anthropic hoặc OpenAI. Cách này cho toàn quyền kiểm soát nhưng trộn lẫn logic điều phối với chi tiết của từng API; mỗi lần đổi nhà cung cấp phải viết lại phần ghép nối, và việc tự phân tích luồng streaming/tool-call dễ lỗi khi chuỗi JSON bị cắt giữa chừng.

**Khung agent đóng gói sẵn.** Các khung agent cấp cao (ví dụ AgentExecutor của LangChain) đóng gói sẵn vòng lặp nên dựng nhanh, nhưng vòng lặp bị giấu sau lớp trừu tượng, làm khó khăn trong việc giới hạn vòng lặp vô hạn, khó lấy sự kiện để stream và khó truy vết từng bước.

**Nền tảng low-code trực quan.** Langflow [3] cho phép dựng luồng agent bằng kéo-thả trên một đồ thị trực quan. Tuy nhiên việc lưu logic dưới dạng đồ thị tĩnh khó ánh xạ sang một engine thực thi theo vòng lặp có thể nhúng vào dịch vụ và phát SSE. Nền tảng kiểu này hợp với tạo mẫu nhanh hơn là làm lõi vận hành.

**Tác tử chạy cục bộ.** Một số công cụ agent chạy cục bộ trên máy người dùng, tối ưu cho thao tác cá nhân nhưng không cung cấp dịch vụ tập trung có API để nhiều client dùng chung và để hệ thống khác tích hợp.

**Lựa chọn của dự án: LangGraph.** Engine dùng LangGraph (Mục 2.1) vì nó mô tả vòng lặp dưới dạng đồ thị — đặt được một nút chặn vòng lặp vô hạn với tham số `max_iter`, đồng thời `astream_events` phát sự kiện ánh xạ gần 1–1



với bộ envelope SSE và với các span quan sát. Nhờ đó đây là lựa chọn thoả mãn đồng thời cả ba yêu cầu nêu trên.

**Vị trí trong nền tảng tổng thể.** Engine này là *tầng thực thi* của một nền tảng AI Agent lớn hơn, song song với nó là *tầng quản trị cấu hình* — nơi định nghĩa và quản lý vòng đời agent, provider/model và skill (vai trò của Module 2/3, Mục 1.2). Hai tầng chia sẻ chung các khái niệm chung như: agent, provider/model, định dạng SKILL.md. Ranh giới giữa chúng được giữ ở tầng định nghĩa qua hai giao diện có thể hoán đổi, nên engine vận hành độc lập được mà vẫn sẵn sàng nhận cấu hình từ tầng quản trị khi tích hợp.

## 2.3. Khảo sát và lựa chọn công nghệ

Lựa chọn công nghệ của engine được chọn theo bốn tiêu chí: (i) hỗ trợ *tool-calling native* và streaming; (ii) khả năng truy vết; (iii) vận hành bất đồng bộ để phục vụ nhiều luồng streaming cùng lúc; và (iv) khả năng tái lập môi trường. Bảng 2.1 liệt kê đầy đủ các thành phần theo từng tầng, kèm phiên bản và vai trò thực tế trong hệ thống.

Bảng 2.1: Lựa chọn công nghệ.

| Công nghệ sử dụng                | Vai trò trong engine                                                                                                                             |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Ngôn ngữ và công cụ build</b> |                                                                                                                                                  |
| Python 3.13                      | Ngôn ngữ chính của engine.                                                                                                                       |
| uv                               | Quản lý gói và môi trường ảo, cài đặt nhanh, khoá phiên bản tất định để tái lập.                                                                 |
| <b>Tầng web / API</b>            |                                                                                                                                                  |
| FastAPI ( $\geq 0.136$ )         | Khung web: khai báo route <code>/api/chat</code> (REST) và <code>/api/chat/stream</code> (SSE), tự sinh tài liệu OpenAPI, tích hợp với Pydantic. |

(còn tiếp)

Bảng 2.1 (tiếp theo)

| Công nghệ sử dụng                      | Vai trò trong engine                                                                                                              |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Uvicorn[standard]                      | Máy chủ chạy FastAPI; hỗ trợ hot-reload khi phát triển.                                                                           |
| sse-starlette                          | Phát Server-Sent Events chuẩn (EventSourceResponse) từ nền Starlette của FastAPI.                                                 |
| <b>Tầng tác tử (agent)</b>             |                                                                                                                                   |
| LangGraph ( $\geq 1.2$ )               | Mô hình hoá vòng lặp agent thành đồ thị máy trạng thái (think/act/max_iter); API astream_events phát sự kiện theo thời gian thực. |
| LangChain Core ( $\geq 1.4$ )          | Trừu tượng thông điệp và bind_tools() (tool-calling native); là giao diện chung để vòng lặp không phụ thuộc nhà cung cấp.         |
| langchain-anthropic / langchain-openai | Adapter tới Anthropic (Claude) và các API tương thích OpenAI (bao gồm Groq) qua cùng một giao diện.                               |
| <b>Trừu tượng dữ liệu và cấu hình</b>  |                                                                                                                                   |
| Pydantic v2 ( $\geq 2.11$ )            | Mô hình hoá và kiểm tra dữ liệu có kiểu: khung envelope sự kiện, schema cấu hình, lược đồ đối số skill.                           |
| pydantic-settings                      | Nạp cấu hình từ biến môi trường (.env) một cách có kiểu, tập trung.                                                               |
| PyYAML                                 | Đọc định nghĩa skill nguyên tử dạng YAML và frontmatter của SKILL.md.                                                             |
| <b>Tầng lưu trữ</b>                    |                                                                                                                                   |
| PostgreSQL 18                          | CSDL quan hệ; kiểu JSONB lưu đối số/kết quả công cụ là then chốt cho tái dựng và kiểm toán; gần với môi trường production.        |
| SQLAlchemy 2.0                         | Lớp truy cập dữ liệu bất đồng bộ; truy vấn an toàn, tách mã nghiệp vụ khỏi SQL thô.                                               |
| asyncpg                                | Driver PostgreSQL hiệu năng cao cho async; quản lý pool kết nối dùng lại giữa các request.                                        |
| <b>Backend dữ liệu của skill</b>       |                                                                                                                                   |
| paramiko                               | Thực thi lệnh chẩn đoán qua SSH cho các skill trên node thật (check_service_status, node-health).                                 |

(còn tiếp)

Bảng 2.1 (tiếp theo)

| Công nghệ sử dụng                                      | Vai trò trong engine                                                                                                                                 |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| ClickHouse                                             | Kho cảnh báo phục vụ truy vấn alarm.                                                                                                                 |
| <b>Quan sát hệ thống</b>                               |                                                                                                                                                      |
| Langfuse (Open-Telemetry)                              | Gắn vết phiên thành <i>trace</i> , mỗi lượt thành <i>span</i> , mỗi lời gọi công cụ thành <i>sub-span</i> ; phân tích chi phí, độ trễ, truy vết lỗi. |
| <b>Giao diện người dùng</b>                            |                                                                                                                                                      |
| Vite + React                                           | Giao diện chat tối thiểu; nhẹ, kiểm soát hoàn toàn việc hiển thị luồng SSE tự thiết kế và hiển thị thẻ suy luận/công cụ.                             |
| <b>Kiểm thử và chất lượng mã (công cụ đã cấu hình)</b> |                                                                                                                                                      |
| pytest + pytest-asyncio                                | Khung kiểm thử cho mã đồng bộ và bất đồng bộ ( <code>asyncio_mode=auto</code> ).                                                                     |
| vcrpy                                                  | Ghi lại rồi phát lại lượt gọi HTTP tới LLM, giúp kiểm thử tắt định và không tốn token.                                                               |
| ruff                                                   | Lint và format mã Python ( <code>line-length=100, target=py313</code> ).                                                                             |
| <b>Hạ tầng</b>                                         |                                                                                                                                                      |
| Docker Compose                                         | Đóng gói môi trường phát triển.                                                                                                                      |

Một vài lựa chọn cần làm rõ thêm:

**LangGraph + LangChain thay vì gọi SDK thô.** Có thể tự viết vòng lặp tool-calling bằng cách gọi trực tiếp SDK của từng nhà cung cấp, nhưng làm vậy sẽ trộn lẫn logic điều phối với chi tiết của từng API. Dùng LangChain Core làm lớp thông điệp chung và LangGraph để mô tả vòng lặp dưới dạng đồ thị giúp tách bạch rõ ràng; quan trọng nhất, `astream_events` phát ra các sự kiện (bắt đầu/kết thúc gọi mô hình, từng token, vòng đời công cụ) ánh xạ gần như 1–1 với bộ envelope SSE tự thiết kế.

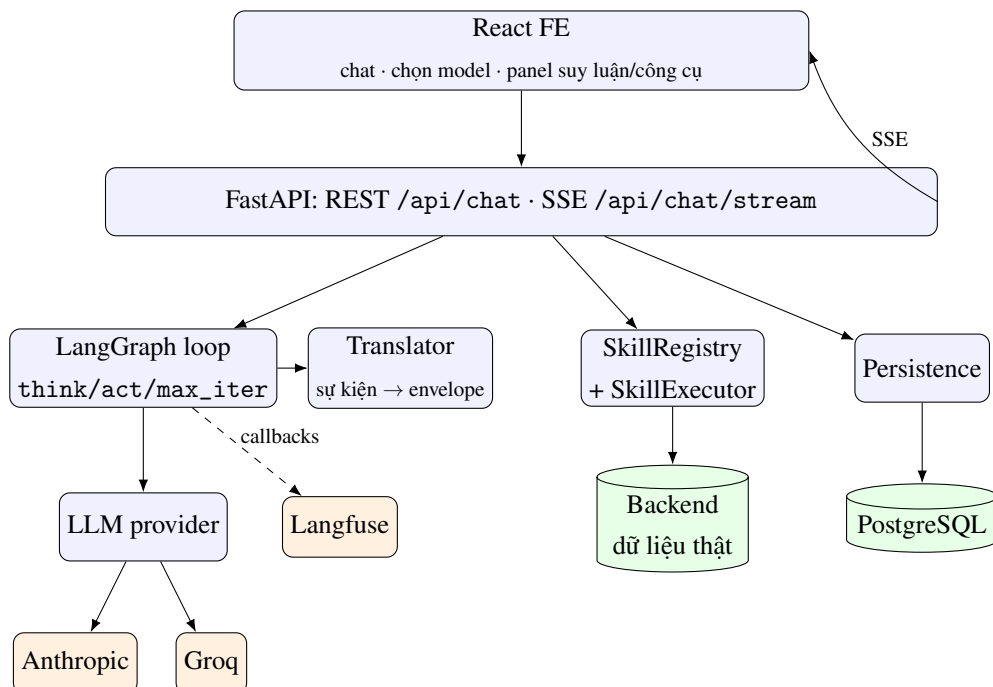
**Ngăn xếp bất đồng bộ đồng nhất.** Toàn bộ đường đi đều async: FastAPI nhận request, LangGraph stream sự kiện, SQLAlchemy + asyncpg ghi vết, httpx gọi

dịch vụ ngoài. Sự đồng nhất này cho phép một tiến trình phục vụ nhiều phiên streaming dài cùng lúc mà không chặn vòng lặp sự kiện.

**Tính tái lập của môi trường.** uv khoá chính xác phiên bản phụ thuộc, còn Docker Compose giúp đóng gói và dựng môi trường, nhờ đó việc phát triển và demo được đảm bảo nhất quán giữa các máy, tránh tình trạng mã chỉ chạy đúng trên một máy cụ thể.

## 2.4. Kiến trúc tổng thể

Hình 2.1 mô tả các thành phần chính của engine và quan hệ giữa chúng. Trình duyệt gửi câu hỏi tới tầng REST/SSE của FastAPI; engine chạy đồ thị LangGraph, gọi LLM qua lớp trừu tượng nhà cung cấp, điều phối skill qua SkillExecutor tới các backend dữ liệu thật, đồng thời phát sự kiện về giao diện và ghi vết xuống PostgreSQL và Langfuse.



Hình 2.1: Kiến trúc tổng thể

Trách nhiệm các thành phần: **tầng REST/SSE** tiếp nhận request và phát

stream; **LangGraph loop** giữ trạng thái mỗi lượt và điều phối vòng lặp; **Translator** dịch sự kiện nội bộ sang envelope; **SkillRegistry/SkillExecutor** nạp định nghĩa skill và thực thi công cụ; **LLM provider** bọc các nhà cung cấp đồng nhất; **Persistence** lưu phiên/tin nhắn/lời gọi công cụ; và **Langfuse** nhận vết quan sát.

**Cô lập với Module 2/3.** Engine chỉ đọc đầu vào qua hai giao diện: `AgentConfigSource` dùng để cấu hình agent và `SkillRegistrySource` dùng để định nghĩa skill. Bản phát triển đọc trực tiếp file cấu hình agent. Sau này, khi cần tích hợp với Module 2/3, chỉ thêm các API endpoint bổ sung mà không cần sửa engine. Việc *thực thi* skill do engine chạy executor trực tiếp.

## 2.5. Đặc tả giao diện lập trình (API)

Engine cung cấp các endpoint HTTP (Bảng 2.2): một cặp REST/SSE cho hội thoại, hai endpoint truy vấn phiên phục vụ tái dựng lịch sử, và một endpoint liệt kê nhà cung cấp/mô hình để giao diện lần bộ kiểm tra hợp lệ dùng chung một nguồn dữ liệu.

Bảng 2.2: Các endpoint HTTP của engine.

| Endpoint               | Mục đích                           | Đầu vào              | Kết quả / lỗi                                                                                             |
|------------------------|------------------------------------|----------------------|-----------------------------------------------------------------------------------------------------------|
| GET /api/health        | Kiểm tra engine còn sống           | —                    | 200: trạng thái và phiên bản.                                                                             |
| GET /api/providers     | Liệt kê provider và model khả dụng | —                    | 200: danh sách provider kèm model.                                                                        |
| GET /api/sessions      | Danh sách phiên gần đây            | —                    | 200: mảng phiên.                                                                                          |
| GET /api/sessions/{id} | Tái dựng lịch sử một phiên         | id (UUID)            | 200: chuỗi tin nhắn để hiển thị; 422 nếu id sai định dạng.                                                |
| POST /api/chat         | Hỏi–đáp đồng bộ (không stream)     | thân JSON (Bảng 2.3) | 200: câu trả lời cuối; lỗi/timeout (mặc định 30 s) được bọc vào trường trả lời.                           |
| POST /api/chat/stream  | Hỏi–đáp dạng streaming             | thân JSON (Bảng 2.3) | 200 text/event-stream: 12 loại sự kiện (Bảng 2.6); lỗi nghiêm trọng phát qua <code>session.error</code> . |

Thân yêu cầu của hai endpoint hội thoại dùng chung một lược đồ (Bảng 2.3).

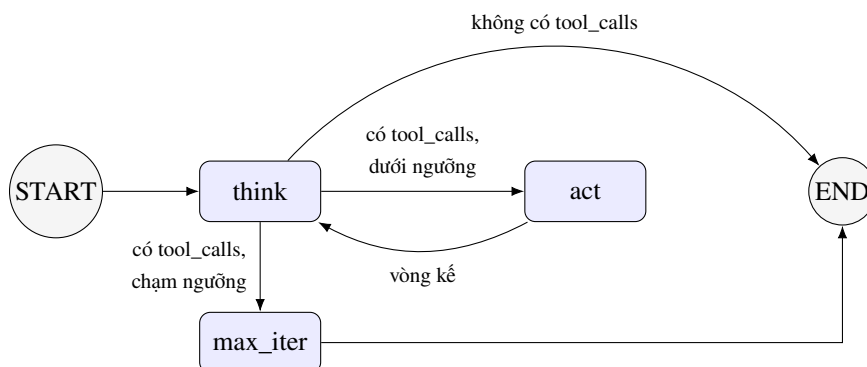
Bảng 2.3: Tham số của `/api/chat` và `/api/chat/stream`.

| Tham số    | Bắt buộc / Mặc định                         | Ý nghĩa                                       |
|------------|---------------------------------------------|-----------------------------------------------|
| question   | Bắt buộc                                    | Câu hỏi ngôn ngữ tự nhiên của kỹ sư vận hành. |
| session_id | Tùy chọn (mặc định: tạo phiên mới)          | UUID phiên để nối tiếp hội thoại nhiều lượt.  |
| provider   | Tùy chọn (mặc định groq)                    | Nhà cung cấp LLM dùng cho lượt này.           |
| model      | Tùy chọn (mặc định llama-3.3-70b-versatile) | Mô hình LLM dùng cho lượt này.                |

Ngoài thân yêu cầu, hai tham số vận hành đặt qua biến môi trường: `MAX_ITERATIONS` (mặc định 8) chặn số vòng lặp tối đa, và `REQUEST_TIMEOUT_SECONDS` (mặc định 30) giới hạn thời gian cho đường `/api/chat` đồng bộ.

## 2.6. Vòng lặp thực thi agent

Lõi của engine là một đồ thị LangGraph ba nút (Hình 2.2). Trạng thái `AgentState` dùng reducer `add_messages` của LangGraph để các mảnh tin nhắn `AIMessageChunk` khi stream được gộp đúng thành một `AIMessage` thống nhất, giữ được mảng `tool_calls`; nếu dùng `operator.add` thông thường thì các mảnh này không được gộp đúng.



Hình 2.2: Đồ thị LangGraph của vòng lặp agent

Cạnh điều kiện `should_continue` là *nơi duy nhất* thực thi việc chặn vòng lặp, theo bảng chân trị  $2 \times 2$  ở Bảng 2.4: chỉ định tuyến sang `act` khi tin nhắn cuối còn lời gọi công cụ và số vòng còn dưới ngưỡng; nếu đã chạm ngưỡng mà vẫn còn lời gọi thì rẽ sang nút `max_iter` để dừng an toàn; còn lại kết thúc lượt.

Bảng 2.4: Bảng chân trị của hàm định tuyến `should_continue`.

| Còn <code>tool_calls</code> | Dưới ngưỡng <code>MAX_ITER</code> | Định tuyến             |
|-----------------------------|-----------------------------------|------------------------|
| Có                          | Có                                | <code>act</code>       |
| Có                          | Không                             | <code>max_iter</code>  |
| Không                       | —                                 | <code>end (END)</code> |

Đoạn mã 2.1 trích phần dựng đồ thị.

```

1 class AgentState(TypedDict):
2     messages: Annotated[list, add_messages] # reducer
3     # merges streamed chunks
4     iterations: int
5
6 def build_graph(llm, tools):
7     llm_with_tools = llm.bind_tools(tools) #
8     # native tool-calling
9
10    async def think_node(state): #
11        # one LLM call per iteration
12        messages = [SystemMessage(content=SYSTEM_PROMPT),
13                    *state["messages"]]
14        response = await llm_with_tools.ainvoke(messages)
15        return {"messages": [response], "iterations":
16                state["iterations"] + 1}
17
18    act_node = ToolNode(tools,
19                        handle_tool_errors=_tool_err) # built-in executor
20
21    def should_continue(state): #
22        # single source of cap enforcement
23        last = state["messages"][-1]

```

```

17         has_tool_calls = bool(getattr(last, "tool_calls",
18                                   None))
19         under_cap = state["iterations"] < MAX_ITER
20         if has_tool_calls and under_cap: return "act"
21         if has_tool_calls:               return "max_iter"
22         return "end"
23
24     builder = StateGraph(AgentState)
25     builder.add_node("think", think_node)
26     builder.add_node("act", act_node)
27     builder.add_node("max_iter", max_iter_node)
28     builder.add_edge(START, "think")
29     builder.add_conditional_edges("think",
30                                   should_continue,
31                                   {"act": "act", "end":
32                                     END, "max_iter":
33                                     "max_iter"})
34
35     builder.add_edge("act", "think")
36     builder.add_edge("max_iter", END)
37     return builder.compile() #
38     compile ONCE at startup

```

Đoạn mã 2.1: Trạng thái và dựng đồ thị LangGraph (agent/loop.py).

Đồ thị được biên dịch *một lần* lúc khởi động và dùng lại giữa các request; ngưỡng MAX\_ITER mặc định bằng 8, đủ cho các luồng vận hành thường cần 2–4 lượt truy vấn và đồng thời giúp chặn vòng lặp vô hạn của agent.

## 2.7. Trừu tượng nhà cung cấp LLM

Để engine không phụ thuộc một nhà cung cấp, mỗi provider được bọc sau một giao diện chung gồm các phương thức `get_chat_model`, `supported_models` và `estimate_cost_usd`, rồi đăng ký trong một registry nạp theo biến môi trường (có khoá API của nhà cung cấp nào thì bật nhà cung cấp đó). Giao diện trả về một đối tượng tương thích `bind_tools()` của LangChain, nên vòng



lập agent không cần biết đang chạy provider nào. Route GET /api/providers tổng hợp danh sách provider và model để dropdown của giao diện và bộ kiểm tra hợp lệ dùng chung một nguồn, tránh trùng lặp. Bảng giá được khai báo cứng theo từng model và cập nhật thủ công; cách làm này chấp nhận rằng số liệu giá có thể không phải mới nhất, thay vì coi như chúng luôn được cập nhật.

## 2.8. Mô hình skill

Engine hỗ trợ hai loại skill, chi tiết xem trong Bảng 2.5.

Bảng 2.5: Hai loại skill trong mô hình.

|            | Atomic                                       | Composite                                                                                             |
|------------|----------------------------------------------|-------------------------------------------------------------------------------------------------------|
| Định nghĩa | File YAML kiểu Function Calling              | Tệp SKILL.md + thư mục scripts/                                                                       |
| Nạp bởi    | FileSkillRegistrySource                      | CompositeSkillSource (đọc front-matter; hỏi orchestrator --schema)                                    |
| Thực thi   | Executor trực tiếp (db_query, ssh...)        | Executor script: chạy orchestrator qua subprocess, mặc định chạy dry-run cho thao tác có tác dụng phụ |
| Ví dụ      | query_device_inventory, check_service_status | node-health-autoremediate, noc-alarm-enrichment                                                       |

SkillExecutor điều phối theo trường executor.type: cùng một dispatcher có một nhánh rẽ giữa skill nguyên tử và skill phức hợp. Đoạn mã 2.2 minh họa một skill nguyên tử thật truy vấn kho thiết bị trên schema alarm\_data (phần mô tả description trong thực tế viết bằng tiếng Việt để LLM đọc và chọn skill; ở đây hiển thị bằng tiếng Anh để giữ đúng định dạng đầu ra).

```
1 - name: query_device_inventory
2   description: "Query the device inventory; optional
3               filters by device_id,
4               device_type, province, or vendor. Returns
               {total, items}."
   executor:
```

```

5     type: db_query
6     target: "${ALARM_PG_URL}"
7     params: [device_id, device_type, province, vendor]
8     sql: >-
9         SELECT d.device_id, d.name, d.device_type, ...
10        FROM alarm_data.device d
11        LEFT JOIN alarm_data.vendor v ON v.vendor_id =
12            d.vendor_id
13        LEFT JOIN alarm_data.station s ON s.station_id =
14            d.station_id
15        WHERE ($1::text IS NULL OR d.device_id = $1) AND ...

```

Đoạn mã 2.2: Skill nguyên tử query\_device\_inventory (rút gọn).

Skill phức hợp node-health minh hoạ rõ giá trị của định dạng SKILL.md: phần đầu (frontmatter) khai báo tên và mô tả để LLM lựa chọn; phần thân là một cây quyết định tự khắc phục sự cố: SSH vào node, đo RAM/CPU, rồi quyết định hành động trên docker theo ngưỡng. Skill này mặc định chạy ở chế độ dry-run cho an toàn, chỉ thực thi thật khi có cờ tương ứng.

## 2.9. Giao thức streaming: envelope sự kiện và Translator

Mỗi sự kiện stream có cùng một khung JSON envelope gồm các trường v, event, ts, session\_id, turn, seq và payload. Trường seq tăng theo phiên để giao diện sắp xếp đúng thứ tự sự kiện. Engine định nghĩa 12 loại sự kiện (Bảng 2.6), bao trùm vòng đời một phiên và một lượt: từ bắt tay phiên, đổi trạng thái, từng token câu trả lời, vòng đời lời gọi công cụ, cho tới kết thúc.

Bảng 2.6: Mười hai loại sự kiện trong envelope SSE.

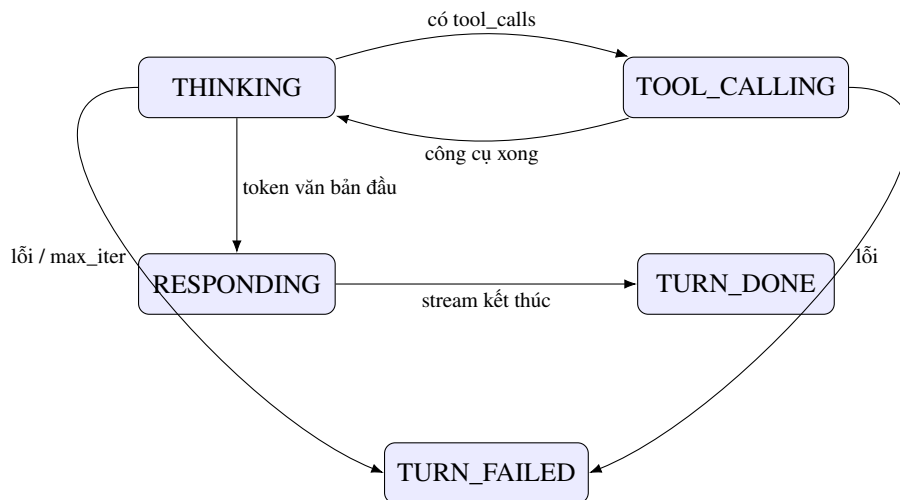
| Sự kiện         | Ý nghĩa                                                                               |
|-----------------|---------------------------------------------------------------------------------------|
| session.started | Bắt tay SSE xong, trước khi đồ thị chạy.                                              |
| turn.started    | Trước vòng đầu của một lượt.                                                          |
| state.changed   | Đổi trạng thái lượt (THINKING / TOOL_CALLING / RESPONDING / TURN_DONE / TURN_FAILED). |

(còn tiếp)

Bảng 2.6 (tiếp theo)

| Sự kiện                          | Ý nghĩa                                                 |
|----------------------------------|---------------------------------------------------------|
| <code>reasoning.delta</code>     | Token suy luận.                                         |
| <code>assistant.delta</code>     | Token câu trả lời cuối.                                 |
| <code>tool_call.requested</code> | LLM yêu cầu gọi công cụ (kèm <code>call_index</code> ). |
| <code>tool_call.running</code>   | Công cụ bắt đầu chạy.                                   |
| <code>tool_call.result</code>    | Công cụ trả kết quả.                                    |
| <code>tool_call.failed</code>    | Công cụ lỗi hoặc quá thời gian.                         |
| <code>turn.finished</code>       | Tin nhắn cuối đã được lưu.                              |
| <code>session.error</code>       | Lỗi nghiêm trọng, đồ thị dừng.                          |
| <code>session.done</code>        | Sự kiện cuối, đóng SSE.                                 |

Translator là *điểm duy nhất* trong mã nguồn chạm vào LangGraph: nó nhận sự kiện từ `astream_events` và phát ra các envelope tương ứng. Quy tắc chính: gặp `tool_call_chunks` thì tích lũy chứ chưa phát; gặp mảnh văn bản đầu tiên thì phát `state.changed` sang RESPONDING (một lần) rồi phát `assistant.delta`; khi mô hình kết thúc mà có `tool_calls` thì gán chỉ số `call_index` xác định và phát `tool_call.requested`. Máy trạng thái cấp lượt được minh hoạ ở Hình 2.3.



Hình 2.3: Máy trạng thái cấp lượt (turn-level FSM).

**Backpressure.** Khi hàng đợi sự kiện gần đầy, chỉ `reasoning.delta` được phép loại bỏ; các sự kiện điều khiển và `assistant.delta` (câu trả lời cuối)

không bao giờ bị mất giúp cho câu trả lời không bị hỏng một cách thảm lạng.

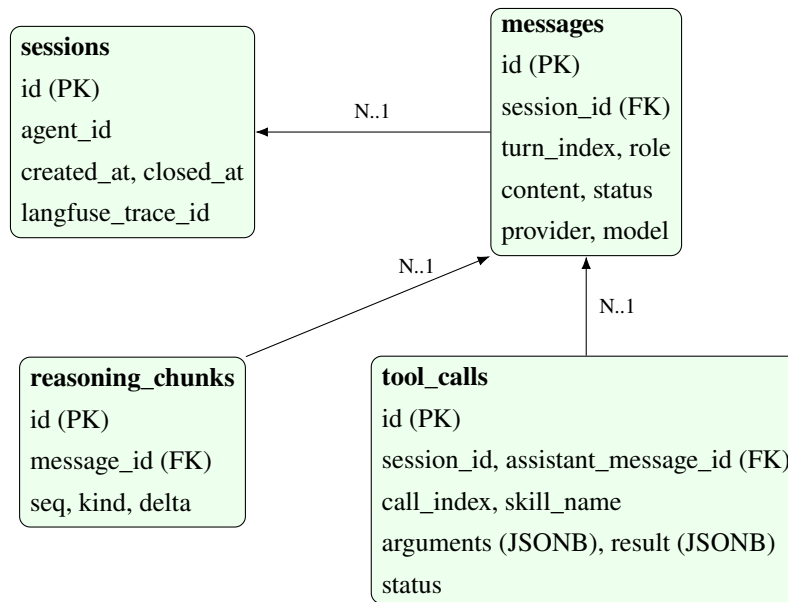
Đoạn mã 2.3 minh hoạ một chuỗi envelope thật mà engine phát ra trong một lượt tra cứu thiết bị: bắt tay phiên, đổi trạng thái, một vòng gọi công cụ, rồi token câu trả lời và đóng lượt. Trường seq tăng đơn điệu để giao diện sắp xếp đúng thứ tự (giá trị ts, session\_id được rút gọn; ký tự tiếng Việt hiển thị không dấu trong đoạn mã).

```
1 event: session.started
2 data: {"v":1,"event":"session.started","ts":"...Z","session_id":"8b1d","turn":1,"seq":1,"payload":{"
  provider":"groq","session_id":"8b1d"}}
3
4 event: turn.started
5 data: {"v":1,"event":"turn.started","ts":"...Z","session_id":"8b1d","turn":1,"seq":2,"payload":{"
  turn_index":1}}
6
7 event: state.changed
8 data: {"v":1,"event":"state.changed","ts":"...Z","session_id":"8b1d","turn":1,"seq":3,"payload":{"
  from":null,"to":"THINKING"}}
9
10 event: state.changed
11 data: {"v":1,"event":"state.changed","ts":"...Z","session_id":"8b1d","turn":1,"seq":4,"payload":{"
  from":"THINKING","to":"TOOL_CALLING"}}
12
13 event: tool_call.requested
14 data: {"v":1,"event":"tool_call.requested","ts":"...Z","session_id":"8b1d","turn":1,"seq":5,"payload
  ":{"call_id":"call_01","call_index":0,"skill_name":"query_device_inventory","arguments":{"
  province":"Ha Noi","device_type":"BTS"}}}
15
16 event: tool_call.result
17 data: {"v":1,"event":"tool_call.result","ts":"...Z","session_id":"8b1d","turn":1,"seq":6,"payload":{"
  call_id":"call_01","skill_name":"query_device_inventory","result":{"total":2,"items":[{"device_id
  ":"BTS-HN-001"}]}}}
18
19 event: state.changed
20 data: {"v":1,"event":"state.changed","ts":"...Z","session_id":"8b1d","turn":1,"seq":7,"payload":{"
  from":"THINKING","to":"RESPONDING"}}
21
22 event: assistant.delta
23 data: {"v":1,"event":"assistant.delta","ts":"...Z","session_id":"8b1d","turn":1,"seq":8,"payload":{"
  delta":"Co 2 thiet bi BTS o Ha Noi."}}
24
25 event: state.changed
26 data: {"v":1,"event":"state.changed","ts":"...Z","session_id":"8b1d","turn":1,"seq":9,"payload":{"
  from":"RESPONDING","to":"TURN_DONE"}}
27
28 event: turn.finished
29 data: {"v":1,"event":"turn.finished","ts":"...Z","session_id":"8b1d","turn":1,"seq":10,"payload":{"
  status":"done"}}
30
31 event: session.done
32 data: {"v":1,"event":"session.done","ts":"...Z","session_id":"8b1d","turn":1,"seq":11,"payload":{"}}
```

Đoạn mã 2.3: Một chuỗi envelope SSE thật của lượt tra cứu thiết bị.

## 2.10. Lưu trữ và kiểm toán

Toàn bộ phiên được lưu trong PostgreSQL để phục vụ tái dựng (Hình 2.4). Đối số và kết quả công cụ được lưu kiểu JSONB [9]. Lược đồ phân tầng độ bền: nhóm *giao dịch* (`sessions`, `messages`, `tool_calls` — phải ghi trước khi người dùng thấy kết quả) và nhóm *best-effort* (`reasoning_chunks` — nếu mất khi nghẽn vẫn chấp nhận được, vì câu trả lời cuối đã nằm an toàn trong `messages.content`).

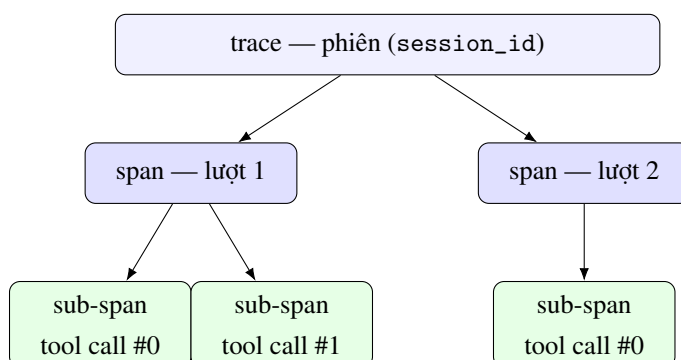


Hình 2.4: Lược đồ dữ liệu chính phục vụ xem lại hội thoại.

## 2.11. Quan sát và truy vết với Langfuse

Một ứng dụng LLM khó gỡ lỗi vì hành vi của nó trải qua nhiều vòng gọi mô hình và gọi công cụ, mỗi vòng phụ thuộc dữ liệu vào/ra của vòng trước. Để quan sát được đường đi này, engine gắn `CallbackHandler` của Langfuse vào mỗi lời gọi đề thị. Cấu trúc truy vết phản ánh đúng cấu trúc hội thoại (Hình 2.5):

mỗi phiên ánh xạ thành một *trace*, mỗi lượt là một *span*, và mỗi lời gọi công cụ là một *sub-span* lồng trong span của lượt đó.



Hình 2.5: Phân cấp truy vết trên Langfuse:  $\text{trace} \rightarrow \text{span} \rightarrow \text{sub-span}$ , song song với lược đồ phiên/lượt/lời gọi công cụ ở Hình 2.4.

Trên mỗi span, bộ tích hợp tự ghi lại dữ liệu vào/ra của lần gọi mô hình, nhà cung cấp và mô hình đã dùng, số token tiêu thụ và độ trễ từng bước. Nhờ đó có thể phân tích chi phí và độ trễ theo từng lượt, đồng thời truy ngược một lời công cụ về đúng `session_id`, lượt và đối số đã gửi — thông tin mà nếu chỉ nhìn câu trả lời cuối thì không thấy được.

Việc gắn vết theo nguyên tắc suy giảm nhẹ nhàng: nếu thiếu khoá hoặc Langfuse không truy cập được, engine vẫn chạy bình thường và chỉ mất khả năng quan sát, không làm hỏng trải nghiệm người dùng (NFR-05).

# Chương 3

## Kết quả thực hiện và đánh giá

Chương này trình bày mục tiêu thực nghiệm, môi trường triển khai, kết quả luồng đầu cuối và đánh giá so với mục tiêu đặt ra. Bức tranh hiện trạng hiện thực hoá so với thiết kế đầy đủ ở Chương 2 được tổng hợp ở Chương 4 để dẫn vào hướng phát triển.

### 3.1. Mục tiêu thực nghiệm

Thực nghiệm nhằm trả lời các câu hỏi nghiên cứu sau:

- **CH1.** Engine có chạy được trọn vẹn một lượt tool-calling đầu cuối — từ câu hỏi tới câu trả lời kèm lời gọi công cụ trên dữ liệu thật — và phát đúng luồng sự kiện streaming hay không?
- **CH2.** Các skill đã hiện thực (truy vấn DB, kiểm tra service qua SSH, skill phức hợp tự khắc phục) có nhận diện và thực thi đúng hay không?
- **CH3.** Dữ liệu phiên (tin nhắn, lời gọi công cụ) có được lưu đầy đủ phục vụ tái dựng hay không?
- **CH4.** Hiệu năng (độ trễ hiển thị token, thời gian một lượt) ở mức nào?

### 3.2. Môi trường và cách triển khai

**Cấu hình:** backend chạy bằng uv, Python 3.13. **Dữ liệu:** skill sử dụng *dữ liệu thật* do Mentor cung cấp, đó là schema `alarm_data` trên PostgreSQL (các bảng `device`, `vendor`, `station`...) và một kho cảnh báo trên ClickHouse. **Nhà cung cấp LLM:** mặc định hiện tại là Groq (model `llama-3.3-70b-versatile`) cho vòng phát triển miễn phí, và Anthropic (model `Claude Opus 4.8`) cho demo.

### 3.3. Kết quả luồng đầu cuối

Luồng đã hoạt động được: người dùng gửi câu hỏi tới POST `/api/chat/stream`; engine tạo phiên, lưu tin nhắn người dùng và một tin nhắn trợ lý ở trạng thái streaming; chạy đồ thị với `astream_events`; Translator dịch từng sự kiện thành envelope, đẩy ra SSE đồng thời ghi các lời gọi công cụ (*requested/result/failed*) xuống DB; khi kết thúc, gộp các `assistant.delta` thành câu trả lời cuối, đánh dấu tin nhắn done và đóng phiên. Ví dụ kịch bản tra cứu thiết bị (`query_device_inventory`) chạy đúng: giao diện hiển thị thẻ công cụ kèm đối số và kết quả truy vấn từ bảng `alarm_data.device`, sau đó là câu trả lời tổng hợp.

### 3.4. Đánh giá

**CH1 — Luồng đầu cuối.** Engine chạy được trọn vẹn một lượt tool-calling đầu cuối và phát đúng luồng sự kiện (10/12 loại; hai loại còn lại đã định nghĩa nhưng chưa phát do chưa bật reasoning/tín hiệu running). Điều này cho thấy tính khả thi của thiết kế kiến trúc và giao thức envelope.

**CH2 — Skill.** Các skill đã hiện thực hoạt động đúng: `query_device_inventory` truy vấn dữ liệu thật từ `alarm_data`; `check_service_status` chạy lệnh qua SSH; skill phức hợp `node-health`; `noc-alarm-enrichment` nạp và thực thi orchestrator.

**CH3 — Tái dựng/kiểm toán.** Trên endpoint `/api/chat/stream`, mỗi phiên, tin nhắn và lời gọi công cụ (kèm đối số và kết quả dạng JSONB) đều được ghi xuống PostgreSQL, đủ dữ liệu để tái dựng lại một lượt hội thoại.

**CH4 — Hiệu năng.**



# Chương 4

## Kết luận

### Kết quả đạt được

Qua báo cáo này, em đã báo cáo thiết kế và bước đầu hiện thực hoá một Agent Execution Engine cho bài toán vận hành mạng viễn thông. Cụ thể, dự án đã:

- Xây dựng vòng lặp tool-calling trên LangGraph (hai nút think/act cùng nút chặn vòng lặp), biên dịch một lần và chạy lại giữa các request.
- Thiết kế lớp trừu tượng đa nhà cung cấp LLM.
- Thiết kế giao thức streaming SSE với envelope 12 loại sự kiện và một bộ Translator dịch sự kiện là nơi duy nhất chạm vào LangGraph.
- Thiết kế mô hình skill lai xử lý cả skill atomic và skill phức hợp. Các executor có thể truy vấn DB, SSH và chạy orchestrator script.
- Lưu trữ phiên/tin nhắn/lời gọi công cụ phục vụ tái dựng và truy vết quan sát qua Langfuse.

Nhờ đó, luồng đầu cuối từ câu hỏi tới streaming câu trả lời kèm lời gọi công cụ trên dữ liệu thật đã hoạt động, cho thấy thiết kế là khả thi.

### Hướng phát triển

Trong thời gian tới, em có kế hoạch hoàn thiện đề tài này với các nội dung sau:

- **Kiểm thử và độ tin cậy:** bổ sung kiểm thử (unit/integration/E2E) theo tháp kiểm thử và đo độ phủ, ưu tiên `should_continue`, Translator và các executor; phát nốt hai sự kiện `reasoning.delta` và `tool_call.running`

để đạt đủ 12 loại.

- **An toàn và vận hành:** bổ sung cơ chế xác nhận hai pha cho skill nhạy cảm ở cấp giao thức; hoàn thiện CORS, xác thực và gắn lưu trữ cho đường /api/chat đồng bộ phục vụ triển khai nhiều client.
- **Mở rộng:** tiến tới mở rộng theo hướng hệ đa tác tử (multi-agent) cho các bài toán vận hành phức tạp hơn.

# Tài liệu tham khảo

- [1] S. Yao, J. Zhao, D. Yu, et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” in *Proc. ICLR*, 2023.
- [2] LangChain, “LangGraph Documentation,” <https://langchain-ai.github.io/langgraph/>, truy cập 2026.
- [3] Langflow, “Langflow Documentation,” <https://docs.langflow.org/>, truy cập 2026.
- [4] Anthropic, “Tool use (function calling),” <https://docs.anthropic.com/>, truy cập 2026.
- [5] OpenAI, “Function calling,” <https://platform.openai.com/docs/guides/function-calling>, truy cập 2026.
- [6] WHATWG, “Server-Sent Events,” HTML Living Standard, <https://html.spec.whatwg.org/multipage/server-sent-events.html>, truy cập 2026.
- [7] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, truy cập 2026.
- [8] Langfuse, “Documentation,” <https://langfuse.com/docs>, truy cập 2026.
- [9] PostgreSQL Global Development Group, “JSON Types (JSONB),” <https://www.postgresql.org/docs/current/datatype-json.html>, truy cập 2026.
- [10] Anthropic, “Agent Skills,” <https://docs.anthropic.com/>, truy cập 2026.

- [11] Pydantic, “Pydantic Documentation,” <https://docs.pydantic.dev/>, truy cập 2026.